

NairaPlate: plans for every recommendation in the product grade assessment

Status: plans for approval. Only the cash-out feature (item 1) is being built. Nothing else here is started. **Date:** 3 October 2026 **Source:** "NairaPlate: Overall Product Grade and Assessment" (3 pages, B+ today at 7.8 out of 10).

0. How to read this

The assessment is an outside opinion. I have not verified its market claims: the food-inflation figure (19.57 percent in August 2026), the competitor claims and the grades themselves are the reviewer's, and I cannot confirm them. I treat them as direction, not fact.

One point in it is out of date. It says historical profit uses current ingredient prices. Since 1 October 2026 every sold line stores what a plate cost at the moment of sale, and nobody signed in can change it. The reviewer most likely read the version of the specification from before I corrected that section (Part 4.5, merged on 3 October). What is still true: older sales have no frozen cost, ingredient prices have no history by date, and batch and wastage costs are worked out on the device. Item 6 plans for those.

How each item is laid out: what the assessment asks for, where we are today (checked against the code and the live database), the plan, how it will be proved, and the decisions I need from you. Sizes (S, M, L) are my estimates, not commitments.

1. The whole picture

#	Recommendation	Assessment stage	Where we are	Size
1	Cash paid out of the drawer, deposit and supplier payment methods	P0 (to reach A-)	Being built. Plan approved, library written, database part next	M
2	Backup restoration proof	P0	Not confirmed. Needs the Supabase dashboard	S
3	Transfer confirmation proved end to end	P0	Code and signature check exist. Never tested with real keys	M
4	Real-user acceptance testing	P0	Scripts written. Not yet run by a person	S to M
5	Safe degraded operation, then a signed offline queue	P1 (to reach A)	Scoped. Late entry approved as first step	M, then L
6	Effective-dated prices and costs for historic profit	P1	Sale-time cost exists since 1 Oct. History by date does not	M
7	Receipt capture	P1	Not started	M
8	Daily exception intelligence	P1	Daily email exists, no exception list	M

#	Recommendation	Assessment stage	Where we are	Size
9	Differentiation, field outcomes and the long-term roadmap	P2	Not started	Ongoing

Suggested order: 1 (in build), then 2, 3 and 4 in parallel because they are mostly you and the field, then 6 (it is the foundation for late entry), then 5, then 8, then 7, then 9.

How this maps to the assessment's stages: items 1 to 4 are what it says takes the product from B+ (7.8) to A- (8.5). Items 5 to 8 are what it says takes it to A (9.0). I will not promise the grades.

2. Item 1. Cash-drawer completeness (P0)

Asked: an append-only cash-paid-out record, payment methods on catering deposits and supplier payments, and expected cash that includes qualifying inflows and outflows.

Today: expected cash counts float, cash sales, cash catering payments and cash debt payments. It leaves out catering deposits (no method saved), cash purchases and supplier payments.

Plan: the approved plan in `NairaPlate_Plan_Cash_Paid_Out_of_Drawer`. In short: an append-only payout ledger per shift, a cashier limit applied to the running total per shift (₦10,000 default, owner-set), owner approval above it, a database rule that stops a shift closing while a request waits, "Paid from the cash drawer" boxes on purchases and supplier payments, and a method on catering deposits.

One part of the assessment that my plan covers only partly: it asks for a payment method on supplier payments. My plan records only whether the cash came from the drawer. If you also want the supplier statement to show cash, transfer or other, that is a small follow-up (a method column and a selector on the supplier payment screen). **Decision 1 below.**

Proof: local database copy, unit tests, a live self-undoing rehearsal, new UAT steps.

3. Item 2. Backup and recovery proof (P0)

Asked: a financial-control product must prove restoration.

Today: unconfirmed. I cannot see the Supabase plan, backup settings or point-in-time recovery from here.

Plan (you do the dashboard steps, I write the checks): 1. **Confirm what exists.** In Supabase, Database, then Backups: note the plan, whether daily backups are on, whether point-in-time recovery is on, and the retention period. Send me a screenshot or the wording. 2. **Set the targets.** Decide how much data you can afford to lose (the recovery point) and how long you can be down (the recovery time). I suggest we write them into the specification. 3. **Restore drill, once now and then every quarter.** - Restore the latest backup, or a point in time from yesterday, into a **separate scratch project**. Never over the live one. - Run `system_health_check.sql` on the restored copy. It must return the same first eight values as live. - Run row counts of the money tables on both and compare (I will give a ready query: orders, order lines, purchases, supplier entries, credit entries, catering payments, cash drawers, payouts, audit lines, stock movements). - Run `rehearsal_demo_kitchen.sql` on the restored copy. It must say ALL CLEAR. - Record the time it took and any steps that failed. 4. **Write the result down.** A one-page record: date, backup used, time to restore, checks passed. It goes into the specification as evidence. 5. **A second copy.** If the plan allows, a weekly export of the money tables to storage outside Supabase. I will plan that once I know the plan tier.

Proved when: a restore into a scratch project passes all three checks and the time is recorded.

Decisions: the recovery point and recovery time you can live with.

4. Item 3. Transfer confirmation end to end (P0)

Asked: automatic transfer confirmation proved end to end.

Today: the payment webhook exists. It looks up the shop from our own reference, verifies the signature with that shop's own secret, and records the payment through one database function. It has unit tests. It has not been run against Monnify with real keys, and this shop has automatic transfers switched off ("Automatic transfers are not switched on for this shop").

Plan: 1. **Sandbox first.** Ask the shop owner to connect Monnify **sandbox** keys to Demo Kitchen on the Payments screen and switch automatic transfers on. 2. **Cases to run, each with a recorded result:** - Exact payment: order goes to Paid, stock reduced once, expected cash unchanged (transfers are not cash). - Short payment: shows "short" with the amount missing, not Paid. - Over payment: recorded with the excess, order Paid. - Duplicate webhook (same event sent twice): the second is ignored, no double credit. - Bad signature and unknown reference: refused or ignored. - A webhook that arrives late, after the request was cancelled. - The webhook for another business's reference. - The customer pays and the webhook never arrives: the screen shows "waiting" and offers the manual fallback. 3. **A signed-request tool.** I will write a small script that sends correctly signed test webhooks using the test shop's secret, so each case can be repeated without paying real money. It must only ever be pointed at Demo Kitchen. 4. **One small real transfer** with live keys in a pilot business, with the owner's agreement, and compare the bank, Monnify and NairaPlate records. 5. **Reconciliation view.** An owner screen that lists transfer requests by status (paid, short, waiting, cancelled) with the amounts, so a missing payment is visible the same day.

Proved when: every case above has a recorded pass and the real transfer reconciles to the kobo.

Decisions: who holds the Monnify account for testing, and whether a small real transfer in a pilot kitchen is acceptable.

5. Item 4. Real-user acceptance (P0)

Asked: operational acceptance with real people, alongside backup restore tests.

Today: the owner script, the first-time-user script with its observer sheet, and a short checklist exist. None has been run by a person yet.

Plan: 1. **Round 1, owner:** run the owner script (about 60 minutes). Send me the failed rows. 2. **Round 2, three people who have never used it:** one cashier, one cook or kitchen staff, one purchaser. I will write short role scripts for the cook and the purchaser in the same style as the cashier one. Each session is watched, with the observer sheet. 3. **Acceptance rules,** agreed before the sessions: for example every critical task finished without help, no task needing more than a set time, no wrong money outcome, and the health check and rehearsal still ALL CLEAR afterwards. I will propose the numbers once you have seen the first session. 4. **Defect list.** Every finding gets a number, a severity (blocks money, blocks work, confusing, cosmetic) and an owner. Anything that touches money is fixed before the next round. 5. **Round 3:** repeat with the fixes. 6. **Two-week parallel pilot** in one real kitchen: staff use the app and the old method side by side, and each day the owner compares them. Measured: shifts whose difference was explained, entries made late, and tasks staff still do on paper.

Proved when: two rounds pass the acceptance rules and the pilot shows no unexplained difference.

Decisions: who the three testers are, and which kitchen will run the pilot.

6. Item 5. Offline operation (P1)

Asked: safe degraded operation first, then a signed local sale queue with device identity, timestamps, sync state, duplicate prevention and controlled conflict review. No unrestricted offline corrections.

Today: no offline mode. The full options are in `NairaPlate_Scoping_Offline_Resilience`. Your decision on 3 October: late entry first, queue later if outage data justifies it, full offline off the roadmap.

Plan in three phases:

Phase 0: safe degraded operation (small). What the app does when the connection drops, without any queue: - A clear "Offline" banner as soon as the connection is lost. - A half-entered sale stays on screen. Nothing is lost by a dropped request, and no sale is reported as saved unless the database confirmed it. - A visible "not saved yet" state with a manual **Try again** button. No hidden automatic resend, because a retry that the database already took must not make a second sale (that needs the idempotency key from Phase 2). - A printable paper-fallback form so staff record sales the same way during an outage, ready for late entry.

Phase 1: late entry (medium, your approved baseline). - A **Late entry** screen for cashiers and owners. Each entry carries the real time the sale happened, the items, how it was paid, and a reason such as "network down". - The database prices each line at the **price in force at the time the sale actually happened**, as you decided. That needs price history by date, so item 6 (dish selling price history) comes first. - An owner approves a batch of late entries. Approved entries become ordinary sales stamped "late entry" with the time delay shown. They count in the shift that was open at the time they happened. If that shift is closed, they are held for the owner with a clear choice. - Stock is reduced when the entry is approved. A shortage in stock is allowed and alerted, as everywhere else. - Late entries cannot be used to correct or cancel sales. Voids and refunds stay online-only. - Every late entry leaves an audit line with who keyed it, who approved it and the delay.

Phase 2: signed offline queue for cash sales (large, only if outage data justifies it). - **Device identity:** each device registers once while online and gets a device id the server knows. Sales from an unknown device are refused. - **Signed sale:** each queued sale has a unique client id (stops duplicates), the device id, the device time and a signature the server can check. - **Sync state:** every queued sale shows waiting, sent, accepted or needs attention. Nothing is dropped silently. - **Server checks at sync:** re-price, re-check the role, check the shift, check the device clock against the server clock, refuse a duplicate client id by returning the first sale. - **Conflict review:** a sale the server will not accept as it stands goes to a "needs attention" list for the owner. The owner accepts, edits (with a reason) or rejects it. - **Scope limits:** cash and split-with-cash sales only; no PIN stored on the device; offline works only while the existing session is valid, for a time the owner sets; no corrections.

Proved when: airplane-mode tests on real phones pass (sell, drop the connection mid-sale, close the browser, restore, double-tap, two devices selling the last plate), plus a database rehearsal for duplicates, a changed price, a closed shift and a bad clock.

Decisions: real outage data from a few kitchens; whether Phase 0 should include the paper-fallback form; the time limit for offline selling if Phase 2 happens.

7. Item 6. Historic profit and effective-dated prices (P1)

Asked: effective-dated ingredient cost history and cost snapshots on sales, batches and wastage.

Today: - Sales since 1 October carry a frozen cost per plate, and the profit and loss uses it. 107 of 110 sold lines (almost all old test sales) have none and are estimated at today's prices. - Batches store an ingredient cost, and wastage entries store a cost, both worked out on the device when logged. - Ingredients keep only the current and the previous price. Their "History" button lists

purchases, not a price over time. - Dish selling prices have no history by date either. A price decision records one change, but an owner can also edit a dish price directly.

Plan: 1. **Ingredient price history (new table).** One row per price change: ingredient, grade, season, price, the moment it took effect, and where it came from (price change, purchase, reversal). Written by the database whenever a price changes, including the existing price function and purchase reversals, so it cannot be skipped. Append-only. 2. **Dish selling price history (new table).** A row each time a dish price changes by any route, written by a database trigger on the dish. This is also what late entry needs. 3. **Backfill, carefully.** The purchases snapshot (price set at, before state) and the "cost changed" audit lines give a partial history for the past. I will fill what is reliable, mark each backfilled row as such, and leave the rest empty rather than guess. 4. **Estimated cost for older sales.** For sold lines with no frozen cost, work out the plate cost from the price in force on the sale date where history exists. The frozen figure is never overwritten. The profit and loss shows how much of the figure is frozen, estimated from history, or estimated at today's price. 5. **Batch and wastage costs worked out by the database,** at log time, from the price in force then, and compared with the figure from the device. A mismatch is refused or flagged. This closes the client-computed gap. 6. **A "cost drift" view for owners:** which dishes cost more now than the price they sell at implies, as inflation moves.

Proved when: the history rows match every price change in a rehearsal, an old sale is re-estimated from history in a test, and the profit and loss for a past week no longer moves when today's prices change.

Decisions: how far back to backfill; whether to refuse or only flag a batch or wastage cost that differs from the database figure.

8. Item 7. Receipt capture (P1)

Asked: receipt capture (named in the assessment's P1 list).

Today: purchases record the amount, supplier, grade and season, and an optional voice transcript. No receipt or invoice can be attached.

Plan: 1. A private storage area per business. Only that business's owner, purchaser and cashier can read it. Nobody can overwrite or delete a file. 2. A **purchase_attachments** record (append-only): the purchase, the file path, the type, the size, who added it and when. A reversal of the purchase leaves the file in place. 3. **Purchases screen:** an **Add receipt photo** button on a purchase, usable from a phone camera, with size and type limits and image shrinking before upload. 4. **Cash payouts from the drawer** can carry a receipt photo too, as the evidence for the money that left. 5.

Optional rule: the owner can ask for a receipt, or a reason when none exists, on cash purchases and payouts above an amount they set. Missing receipts then show in the daily exception list (item 8). 6. **Not now:** reading the amounts from the photo automatically. I would not trust that for a financial record until the manual flow is proven.

Proved when: a person on a phone adds a photo, the owner sees it, another business cannot, and a file cannot be replaced.

Decisions: the size limit, how long files are kept, and whether the receipt rule should exist at all.

9. Item 8. Daily exception intelligence (P1)

Asked: a daily view that tells the owner what needs attention.

Today: the daily summary email carries the profit and loss and the cost check. It lists no exceptions.

Plan: 1. **A read-only database function** that returns the day's exceptions for one business, from data we already hold, each with who, what, how much and a link: - A shift closed short or over, or closed without a count. - Voids and refunds above a count or amount, by cashier. - Reversals of any kind (purchase, payment, payout, batch, price decision, debt entry). - Cash payout requests waiting,

declined, or reversed. - Stock counts with differences, stock below zero, entries "changed by hand" (should be zero). - A dish price or ingredient price change above a set percentage. - A batch that cost much more per plate than the recipe says. - Customer credit written off, and debts passing a set age. - Cash purchases or payouts with no receipt, once item 7 exists. - Sales entered late, once item 5 exists. 2. **Thresholds per business** with sensible defaults, set by the owner, in one small settings table that only a function can change. 3. **An Exceptions screen** for the owner, newest first, with **Acknowledge** and a note. Acknowledging is itself an entry: it is never deleted. 4. **The daily email gets an Exceptions section** with the top items and a link, and an "all clear" line when there are none. 5. **No claims beyond the data.** The list says what happened. It does not guess at fraud.

Proved when: a rehearsal creates each exception and the function lists each one exactly once, and the email shows them.

Decisions: the default thresholds, and who besides the owner may see the list.

10. Item 9. Differentiation, outcomes and the long term (P2)

Asked: differentiate through verifiable owner control, not generic feature claims; reach multi-branch controls, kitchen flow, delivery reconciliation, accounting integrations and measured customer outcomes.

Plan: 1. **Make "every naira explained" visible.** At shift close and at day end, show the owner a single statement: the expected cash broken into its parts, what was counted, and the unexplained difference, plus stock movements with no reason (should be zero). The assessment's own test: can an owner finish every shift knowing every naira and every important stock movement has an explanation? This is item 1 plus item 8, shown in one place. 2. **Measure outcomes in the pilot (item 4):** shortages per week, stock difference percent, entries made late, time to close a shift, before and after. Only real numbers go into marketing. 3. **Long-term roadmap, each needing its own plan before any build:** - **Multi-branch:** a business group above single businesses with branch roles. This changes how tenancy works and is the largest item. Not before the pilot results. - **Kitchen flow:** a read-only kitchen display of open orders first, then batch-aware prep. - **Delivery reconciliation:** channel payouts already compare what a platform paid against sales. Next is importing the platform's statement and matching lines. - **Accounting export:** a clean export of sales, purchases, supplier and customer balances to a spreadsheet, then to accounting packages. - **Compliance:** invoicing and tax requirements need a source I have not checked. I will not state any rule until you give me one.

11. Decisions I need from you

1. **Supplier payments:** do you want a payment method (cash, transfer, other) on supplier payments as well as the "Paid from the cash drawer" box?
2. **Backups:** the recovery point and recovery time you can accept, and the plan tier.
3. **Monnify:** who holds the test account, and may a small real transfer be done in a pilot kitchen?
4. **Testers and pilot:** the three first-time testers and the pilot kitchen.
5. **Offline:** real outage data from a few kitchens; whether to build Phase 0 (degraded operation) now.
6. **Historic costing:** how far back to backfill, and whether a batch or wastage cost that differs from the database figure is refused or only flagged.
7. **Receipts:** size limit, retention, and whether to ask for a receipt above an amount.
8. **Exceptions:** default thresholds.
9. **Order of work:** is the order in section 1 right, after the cash-out build?

12. Where the cash-out build stands

The branch `ccr-cash-out` has the payout library and its 17 tests (committed). The database part (tables, functions, close guard, catering method), the screens and the rehearsal are next. I have paused it while you read this. Say "resume" and I carry on with the approved plan.